KUBERNETES INDIVIDUAL PRACTICAL ASSIGNMENT

# Kubernetes Workload and Service Management

Project documentation for the **TripPlanner** (Travel Booking) scenario: components, manifests, structure and how they depend on each other.

| STUDENT | APPLICATION | NAMESPACE |
|---|---|---|
| 17 · Ahnaf | tripplanner | travel-dev |

| DEPLOYMENT REPLICAS | CONTAINER PORT | NODEPORT |
|---|---|---|
| 3 | 8017 | 30097 |


1 Assignment overview
2 Project structure & file responsibilities
3 System architecture
4 Dependencies between files
5 01-pod.yaml
6 02-deployment.yaml
7 03-service.yaml
8 04-statefulset.yaml
9 05-daemonset.yaml
10 06-job.yaml & 07-cronjob.yaml
11 Deployment order & verification
12 Compliance review & recommended fixes


The YAML listings in this document are copied verbatim from the repository (branch `main`, commit `c42e99e`). No project file was modified. Code marked **"Suggested"** is not in the repository; it shows how the missing items could be completed.

# 1 Assignment overview

The assignment asks for seven Kubernetes resources, each in its own YAML manifest, deployed into the student's own namespace and verified on a running cluster. Every resource name must start with the student's name as a prefix; this project uses `ahnaf-tripplanner-…`.

## Assigned configuration (Student 17)

| ITEM | REQUIRED VALUE | VALUE IN THE MANIFESTS | STATUS |
|---|---|---|---|
| Project scenario | Travel Booking | TripPlanner | MATCH |
| Application name | `tripplanner` | `app: tripplanner` labels, `ahnaf-tripplanner-*` names | MATCH |
| Namespace | `travel-dev` | `travel-dev` (missing on the headless Service, see §7) | MOSTLY |
| Deployment replicas | 3 | `replicas: 3` | MATCH |
| Container port | 8017 | `containerPort: 8017`, and Nginx is rewritten to listen on 8017 | MATCH |
| NodePort | 30097 | `nodePort: 30097` | MATCH |

## The seven questions at a glance

| # | RESOURCE | FILE | PURPOSE IN THE TRIPPLANNER SYSTEM |
|---|---|---|---|
| 1 | Pod | `01-pod.yaml` | A single standalone Nginx web-server Pod, the smallest deployable unit. |
| 2 | Deployment | `02-deployment.yaml` | The front-end web tier: 3 identical Nginx replicas that can be scaled and updated with a rolling update. |
| 3 | Service ×2 | `03-service.yaml` | A NodePort Service that exposes the web tier outside the cluster, and a headless Service that gives the database Pods stable DNS names. |
| 4 | StatefulSet | `04-statefulset.yaml` | The PostgreSQL database tier: 2 Pods with stable names and one persistent volume each. |
| 5 | DaemonSet | `05-daemonset.yaml` | A log-collector agent that runs exactly one Pod on every eligible node. |
| 6 | Job | `06-job.yaml` | A one-off database cleanup task. **The file is currently empty.** |
| 7 | CronJob | `07-cronjob.yaml` | A database maintenance task that runs every 5 minutes. **The file is currently empty.** |

## Core Kubernetes concepts used

### Labels & selectors

Key/value tags on objects. Controllers (Deployment, StatefulSet, DaemonSet) and Services find “their” Pods by matching a `selector` against Pod `labels`. This is the main link between the files.

### Controllers

Deployment, StatefulSet, DaemonSet, Job and CronJob all keep a set of Pods in a desired state. They differ in identity (random vs. ordinal names), placement (per node), and lifetime (long-running vs. run-to-completion).

### Services

A stable virtual IP and DNS name in front of a changing set of Pods. **NodePort** also opens a port on every node; **headless** (`clusterIP: None`) skips the virtual IP and returns the Pod IPs directly.

### Persistent storage

A `volumeClaimTemplate` makes one PersistentVolumeClaim per StatefulSet replica. The cluster's default StorageClass provisions a matching PersistentVolume automatically.

### Resource requests & limits

`requests` is what the scheduler reserves on a node. `limits` is the hard cap: CPU is throttled at the limit, and memory over the limit gets the container OOM-killed. `100m` means 0.1 of a CPU core.

### Namespaces

A logical partition of the cluster. All TripPlanner objects belong in `travel-dev`, which keeps them separate from other students' resources.

# 2 Project structure & file responsibilities

The repository is flat: one manifest per question, numbered in the order they should be read. There is no application source code; the "application" is the stock `nginx:1.27`, `postgres:16` and `busybox:1.36` images, configured by the manifests.

```
Kubernetes Workload and Service Management/   (git repo, branch main)
│
├── 01-pod.yaml            Pod          ahnaf-tripplanner-pod
├── 02-deployment.yaml     Deployment   ahnaf-tripplanner-deployment
├── 03-service.yaml        Service      ahnaf-tripplanner-service      (NodePort)
│                          Service      ahnaf-tripplanner-db-service   (Headless)
├── 04-statefulset.yaml    StatefulSet  ahnaf-tripplanner-db
├── 05-daemonset.yaml      DaemonSet    ahnaf-tripplanner-daemonset
├── 06-job.yaml            (empty)
└── 07-cronjob.yaml        (empty)
```

### Object ownership chain

Kubernetes controllers create child objects and record themselves in each child's `ownerReferences`. Deleting a parent removes its children. PVCs created by a StatefulSet are the exception: they are kept by default, so the data survives.

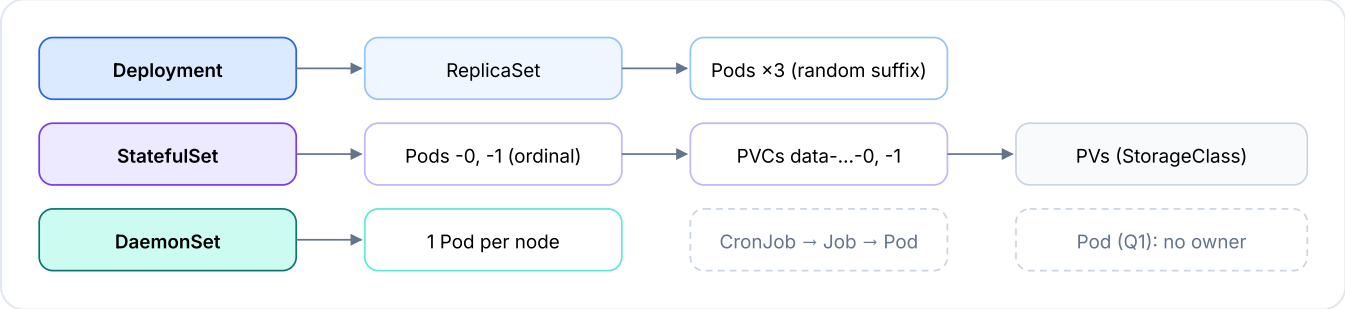



Figure 1. Which objects each manifest ultimately produces. Dashed boxes are not yet created by the repository.

### Who is responsible for what

| FILE | OBJECTS CREATED | RESPONSIBILITY | DEPENDS ON |
|---|---|---|---|
| `01-pod.yaml` | 1 Pod ( `nginx:1.27` ) | Runs one unmanaged web-server container on port 8017. If it dies or its node fails, nothing recreates it. That is the point of contrast with the Deployment. | Namespace `travel-dev` |
| `02-deployment.yaml` | Deployment → ReplicaSet → 3 Pods | Owns the web tier. It keeps exactly 3 replicas running, replaces failed Pods, and handles scaling and rolling updates. | Namespace |
| `03-service.yaml` (doc 1) | NodePort Service | Gives the web Pods one stable address. Traffic to `<NodeIP>:30097` or `service:80` is load-balanced across all Pods labelled `app=tripplanner` on port 8017. | Pods from 01 & 02 (by label) |

| | | | |
|---|---|---|---|
| `03-service.yaml` (doc 2) | Headless Service | Has no virtual IP. It gives each database Pod a DNS record such as `ahnaf-tripplanner-db-0.ahnaf-tripplanner-db-service`. | Pods from 04 (by label) |
| `04-statefulset.yaml` | StatefulSet → 2 Pods + 2 PVCs | Owns the database tier. It creates `…-db-0` then `…-db-1` in order, binds each to its own 1 GiB volume, and recreates a deleted Pod with the same name and volume. | Headless Service, StorageClass |
| `05-daemonset.yaml` | DaemonSet → 1 Pod per node | Node-level agent: a BusyBox loop that writes a heartbeat line every 60 s. New nodes get a Pod automatically. | Namespace |
| `06-job.yaml` | — (empty) | Should run a one-time simulated DB cleanup to completion. | Namespace |
| `07-cronjob.yaml` | — (empty) | Should create a new Job every 5 minutes for simulated DB maintenance. | Namespace |

# 3 System architecture

The manifests together describe a small two-tier travel-booking system: a stateless web tier reachable from outside the cluster, a stateful database tier with persistent disks, and a node-level logging agent.

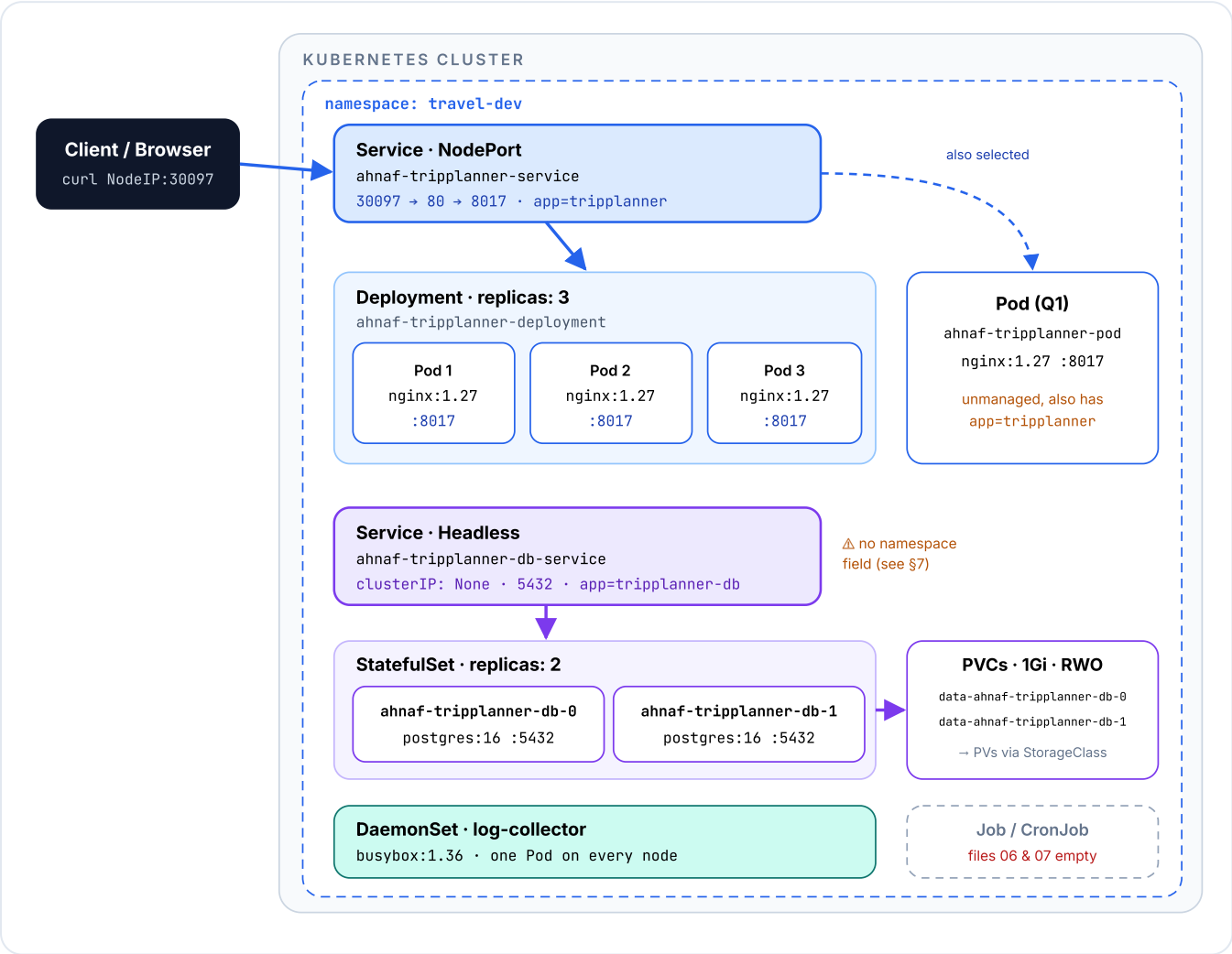



Figure 2. Runtime architecture of the TripPlanner workloads in namespace `travel-dev`. Solid arrows show traffic and selection; the dashed arrow shows the standalone Pod also matching the NodePort selector.

## Request path: how a browser reaches Nginx on port 8017

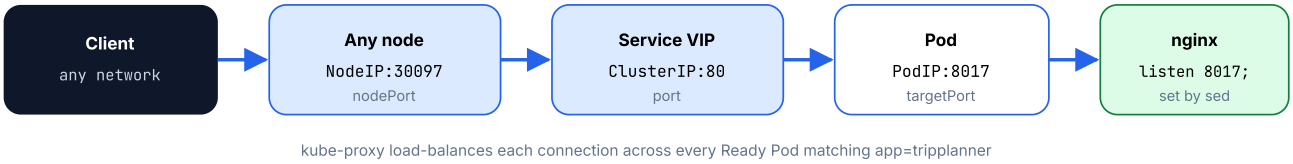



Figure 3. The three port numbers in `03-service.yaml` and how they connect to `containerPort` in 01/02.

# 4 Dependencies between files

Kubernetes manifests do not import each other. They are linked by **names** (`namespace`, `serviceName`) and by **label selectors**. If one side is misspelled, there is no compile error; the link simply never forms. The diagram below shows every such link in this project.

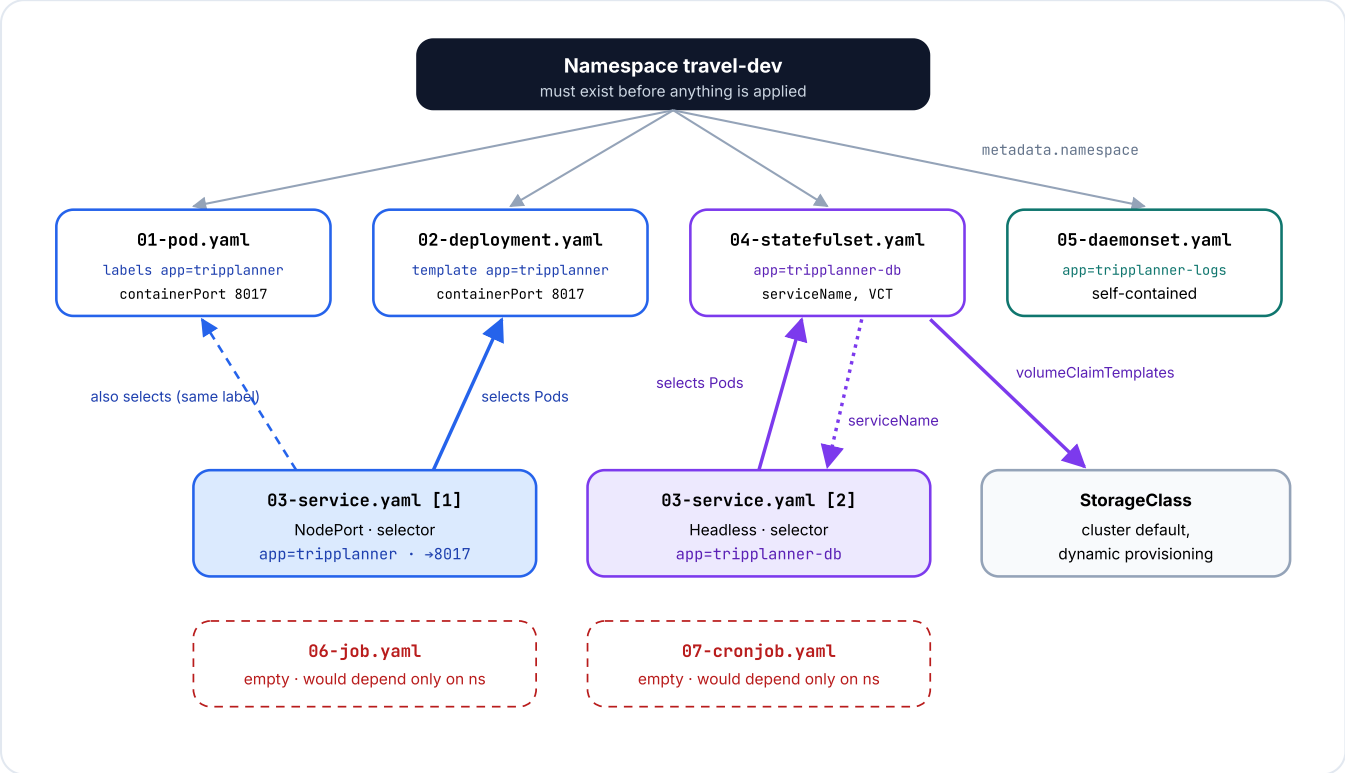



Figure 4. Dependency graph. An arrow means “refers to / needs”. Blue is the web tier, violet the database tier, teal the node agent.

## Dependency details

| FROM | TO | LINK FIELD | WHAT BREAKS IF IT IS WRONG |
|---|---|---|---|
| All files | Namespace `travel-dev` | `metadata.namespace` | `kubectl apply` fails with *namespaces "travel-dev" not found*. The namespace is not defined in any file, so it must be created first. |
| 02 Deployment `selector` | 02 Pod template `labels` | `app: tripplanner` | The API server rejects the Deployment: *selector does not match template labels*. |
| 03 NodePort `selector` | Pods of 01 and 02 | `app: tripplanner` | The Service has no endpoints and `curl :30097` hangs or is refused. |
| 03 NodePort `targetPort` | 01/02 `containerPort` + Nginx `listen` | `8017` | Connections are refused. Nginx must actually listen on 8017, which is why the `sed` command is required. |
| 04 `serviceName` | 03 headless Service | `ahnaf-tripplanner-db-service` | Pods still start, but the per-Pod DNS names (`…-db-0.ahnaf-tripplanner-db-service`) do not resolve. The Service must be in the **same namespace** as the StatefulSet. |

| | | | |
|---|---|---|---|
| 03 headless `selector` | 04 Pod template `labels` | `app: tripplanner-db` | DNS returns no addresses for the database Pods. |
| 04 `volumeMounts.name` | 04 `volumeClaimTemplates.metadata.name` | `data` | The StatefulSet is rejected or the Pod cannot start because the volume is not found. |
| 04 `volumeClaimTemplates` | Cluster StorageClass | (default class) | PVCs stay `Pending`, so the Pods stay `Pending`. |
| 04 env `PGDATA` | 04 `mountPath` | `/var/lib/postgresql/data/pgdata` | Using the mount root directly can fail with *directory not empty* (`lost+found`). The sub-directory avoids that. |

**Why the label values are deliberately different**

The web tier uses `app=tripplanner`, the database `app=tripplanner-db` and the agent `app=tripplanner-logs`. Because they differ, the NodePort Service never sends web traffic to PostgreSQL or to the log agent, and each controller only manages its own Pods.

01-pod.yaml QUESTION 1 · POD

# 5 Standalone Pod

Creates one Nginx container that serves on the assigned port 8017. A bare Pod has no controller, so Kubernetes will not restart it on another node if it is deleted or its node fails.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: ahnaf-tripplanner-pod
  namespace: travel-dev
  labels:
    app: tripplanner
    environment: development
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      env:
        - name: NGINX_PORT
          value: "8017"
      command: ["/bin/sh","-c"]
      args:
        - sed -i "s/listen  *80;/listen ${NGINX_PORT};/g" /etc/nginx/conf.d/default.conf &&
nginx -g "daemon off;"
      ports:
       - containerPort: 8017
      resources:
        requests:
          memory: "64Mi"
          cpu: "250m"
        limits:
          memory: "128Mi"
          cpu: "500m"

```

## Field-by-field explanation

| FIELD | VALUE | MEANING |
|---|---|---|
| `apiVersion` / `kind` | `v1` / `Pod` | Core API group; the object type to create. |
| `metadata.name` | `ahnaf-tripplanner-pod` | Unique name with the required student prefix. |
| `metadata.namespace` | `travel-dev` | The assigned namespace. |
| `metadata.labels` | `app`, `environment` | Required labels. `app=tripplanner` also makes this Pod a backend of the NodePort Service. |
| `image` | `nginx:1.27` | Required image, pinned to a minor version. |
| `env.NGINX_PORT` | `"8017"` | The port as an environment variable, so the shell command can read it. |

| | | |
|---|---|---|
| `command` + `args` | `/bin/sh -c "sed … && nginx -g 'daemon off;'"` | Overrides the image entrypoint. `sed` rewrites `listen 80;` in `/etc/nginx/conf.d/default.conf` to `listen 8017;`, then starts Nginx in the foreground so the container keeps running. |
| `ports.containerPort` | `8017` | Documents the listening port. It does not open the port itself; the `sed` rewrite does. |
| `resources.requests` | 64Mi / 250m | Reserved by the scheduler: 64 MiB of memory and a quarter of a CPU core. |
| `resources.limits` | 128Mi / 500m | Hard cap. Memory above it gets the container OOM-killed; CPU above it is throttled. |

**How the port rewrite works**

The regex `listen *80;` is "listen", then one or more spaces, then `80;`. It matches the IPv4 line `listen 80;` in the stock config. The IPv6 line `listen [::]:80;` does not match and stays on port 80, which does no harm. Because the substitution runs before Nginx starts, the server only binds the new port.

### Verify

```
kubectl apply -f 01-pod.yaml
kubectl get pod ahnaf-tripplanner-pod -n travel-dev -o wide      # STATUS = Running
kubectl exec -n travel-dev ahnaf-tripplanner-pod -- grep listen /etc/nginx/conf.d/default.conf
kubectl port-forward -n travel-dev pod/ahnaf-tripplanner-pod 8017:8017   # then open
http://localhost:8017
```

02-deployment.yaml QUESTION 2 · DEPLOYMENT

# 6 Deployment (web tier)

Runs the same Nginx container as the Pod, but managed: a ReplicaSet keeps 3 copies alive, and the Deployment swaps ReplicaSets gradually when the template changes (a rolling update).

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ahnaf-tripplanner-deployment
  namespace: travel-dev
  labels:
    app: tripplanner
    environment: development
spec:
  replicas: 3
  selector:
    matchLabels:
      app: tripplanner
  template:
    metadata:
      labels:
        app: tripplanner
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          env:
            - name: NGINX_PORT
              value: "8017"
          command: ["/bin/sh", "-c"]
          args:
            - sed -i "s/listen *80;/listen ${NGINX_PORT};/g" /etc/nginx/conf.d/default.conf &&
nginx -g "daemon off;"
          ports:
            - containerPort: 8017
          resources:
            requests:
              memory: "64Mi"
              cpu: "100m"
            limits:
              memory: "128Mi"
              cpu: "500m"
```

## Field-by-field explanation

| FIELD | VALUE | MEANING |
|---|---|---|
| `apiVersion` | `apps/v1` | Deployments live in the `apps` API group. |
| `spec.replicas` | `3` | Assigned replica count for Student 17. |
| `spec.selector.matchLabels` | `app: tripplanner` | Tells the Deployment which Pods it owns. It must match the template labels and cannot be changed after creation. |

| | | |
|---|---|---|
| `spec.template` | Pod template | The blueprint for each replica. Any change here, such as a new image or env var, starts a rolling update. |
| container / env / command | same as 01 | Same `sed` port rewrite. The regex here is `listen *80;` (zero or more spaces), which is equally valid. |
| `resources` | req 64Mi/100m · lim 128Mi/500m | Per-replica values, so 3 replicas reserve 192 MiB and 0.3 CPU in total. |
| `strategy` (default) | RollingUpdate 25% / 25% | Not set, so the defaults apply: at most 1 extra Pod (maxSurge 25% rounds up) and at most 0 unavailable (maxUnavailable 25% rounds down) during an update. |

**Note**

`metadata.labels.environment` is set on the Deployment object but not on the Pod template, so the 3 replicas carry only `app=tripplanner`. That is valid, but `kubectl get pods -l environment=development` will not list them.

### Scaling (requirement 6)

```
kubectl scale deployment ahnaf-tripplanner-deployment --replicas=5 -n travel-dev
kubectl get deployment ahnaf-tripplanner-deployment -n travel-dev    # READY 5/5
kubectl scale deployment ahnaf-tripplanner-deployment --replicas=3 -n travel-dev   # back to
assigned value
```

### Rolling update (requirement 7)

```
# change the Pod template without changing the required image family
kubectl set image deployment/ahnaf-tripplanner-deployment nginx=nginx:1.27-alpine -n travel-dev
#   or: kubectl set env deployment/ahnaf-tripplanner-deployment RELEASE=v2 -n travel-dev
kubectl rollout status  deployment/ahnaf-tripplanner-deployment -n travel-dev
kubectl rollout history deployment/ahnaf-tripplanner-deployment -n travel-dev
kubectl rollout undo    deployment/ahnaf-tripplanner-deployment -n travel-dev   # optional rollback
```

03-service.yaml QUESTION 3 · SERVICES

# 7 NodePort & Headless Services

One file, two YAML documents separated by `---`. The first exposes the web tier; the second is the network identity the StatefulSet requires.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: ahnaf-tripplanner-service
  namespace: travel-dev
  labels:
    app: tripplanner
    environment: development
spec:
  type: NodePort
  selector:
    app: tripplanner
  ports:
  - port: 80
    targetPort: 8017
    nodePort: 30097
---
apiVersion: v1
kind: Service
metadata:
  name: ahnaf-tripplanner-db-service
  labels:
    app: tripplanner-db
spec:
  clusterIP: None
  selector:
    app: tripplanner-db
  ports:
  - port: 5432
    name: postgres
```

## Document 1: NodePort Service

| FIELD | VALUE | MEANING |
|---|---|---|
| `type` | `NodePort` | Allocates a ClusterIP and opens the same port on every node. |
| `selector` | `app: tripplanner` | Backends are every Ready Pod with this label in `travel-dev`: the 3 Deployment replicas **and** the standalone Pod from Q1, so 4 endpoints in total. |
| `port` | `80` | The Service's own port, used inside the cluster (`http://ahnaf-tripplanner-service`). |
| `targetPort` | `8017` | The Pod port that traffic is forwarded to. |
| `nodePort` | `30097` | Assigned external port, within the allowed 30000–32767 range. |

### Document 2: Headless Service

| FIELD | VALUE | MEANING |
|---|---|---|
| `clusterIP` | `None` | Makes the Service headless: DNS returns the Pod IPs directly and gives each StatefulSet Pod its own A record. |
| `selector` | `app: tripplanner-db` | Matches the StatefulSet Pods. |
| `ports` | `5432`, name `postgres` | The PostgreSQL port. The name matches the StatefulSet's named container port. |

**Issue: the headless Service has no namespace**

Document 2 has no `metadata.namespace`. Running `kubectl apply -f 03-service.yaml` creates it in the **default** namespace (or whichever namespace the kubectl context uses), while the StatefulSet that refers to it is in `travel-dev`. Pod DNS such as `ahnaf-tripplanner-db-0.ahnaf-tripplanner-db-service.travel-dev.svc.cluster.local` would then not resolve. To work around it without editing the file, apply it with `-n travel-dev`. Document 1 already declares `travel-dev`, so the two agree. Its `labels` also omit `environment`. A suggested fix is in §12.

### Verify

```
kubectl apply -f 03-service.yaml -n travel-dev
kubectl get svc -n travel-dev
kubectl get endpoints ahnaf-tripplanner-service -n travel-dev     # 4 IPs on :8017
curl http://<node-ip>:30097                                          # Nginx welcome page
minikube service ahnaf-tripplanner-service -n travel-dev --url     # if using minikube
```

04-statefulset.yaml QUESTION 4 · STATEFULSET

# 8 StatefulSet (database tier)

Runs PostgreSQL 16 as two Pods with fixed ordinal names and one dedicated persistent volume each. A deleted Pod comes back with the same name, the same DNS record and the same disk.

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: ahnaf-tripplanner-db
  namespace: travel-dev
  labels:
    app: tripplanner-db
    environment: development
spec:
  serviceName: ahnaf-tripplanner-db-service
  replicas: 2
  selector:
    matchLabels:
      app: tripplanner-db
  template:
    metadata:
      labels:
        app: tripplanner-db
        environment: development
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
              name: postgres
          env:
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
            - name: POSTGRES_DB
              value: "postgress"
            - name: POSTGRES_USER
              value: "postgress"
            - name: POSTGRES_PASSWORD
              value: "postgress"
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data/
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "256Mi"
              cpu: "250m"
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
```

## Field-by-field explanation

| FIELD | VALUE | MEANING |
| --- | --- | --- |

| | | |
|---|---|---|
| `metadata.name` | `ahnaf-tripplanner-db` | Becomes the Pod name prefix: `ahnaf-tripplanner-db-0` and `-1`. |
| `spec.serviceName` | `ahnaf-tripplanner-db-service` | The headless Service (03, document 2) that controls the Pods' DNS domain. |
| `spec.replicas` | `2` | Meets "at least two". The default `OrderedReady` policy creates `-0` first, then `-1` once `-0` is Ready. |
| `selector` / template labels | `app: tripplanner-db` | Must match; shared with the headless Service selector. |
| `image` / `ports` | `postgres:16` / `5432` `"postgres"` | Required image; named port. |
| `env PGDATA` | `…/data/pgdata` | Puts the database cluster in a sub-folder of the mounted volume. |
| `env POSTGRES_DB/USER/PASSWORD` | `"postgress"` | Initial database credentials, currently in **plain text**. |
| `volumeMounts` | `data` → `/var/lib/postgresql/data/` | Mounts the per-Pod PVC. |
| `volumeClaimTemplates` | `data`, RWO, 1Gi | Template for one PVC per replica, named `<template>-<statefulset>-<ordinal>`: `data-ahnaf-tripplanner-db-0` and `data-ahnaf-tripplanner-db-1`. |
| `resources` | req 128Mi/100m · lim 256Mi/250m | Per database Pod. |

**Issue: credentials are not in a Secret**
The assignment says "Use a Kubernetes Secret for database credentials." The manifest sets them as literal `env` values, and no Secret object exists in the project. A suggested Secret and `envFrom` change are in §12.

**Good to know**
The two replicas are **independent** PostgreSQL servers; there is no replication between them. The assignment explicitly allows this ("You do not need to build a complete production database").

### Verify: identity, self-healing and persistence

```
kubectl apply -f 04-statefulset.yaml
kubectl get sts,pods,pvc -n travel-dev -l app=tripplanner-db   # db-0, db-1 · 2 PVCs Bound

# write data, delete the Pod, read it back
kubectl exec -n travel-dev ahnaf-tripplanner-db-0 -- psql -U postgress -d postgress \
  -c "CREATE TABLE trips(id int, city text); INSERT INTO trips VALUES (1,'Cox''s Bazar');"
kubectl delete pod ahnaf-tripplanner-db-0 -n travel-dev
kubectl get pods -n travel-dev -w                              # comes back as ahnaf-tripplanner-db-
0
kubectl exec -n travel-dev ahnaf-tripplanner-db-0 -- psql -U postgress -d postgress -c "SELECT *
FROM trips;"

# stable DNS (needs the headless Service in travel-dev)
kubectl run dns-test -n travel-dev --rm -it --image=busybox:1.36 --restart=Never -- \
  nslookup ahnaf-tripplanner-db-0.ahnaf-tripplanner-db-service
```

05-daemonset.yaml QUESTION 5 · DAEMONSET

# 9 DaemonSet (node log agent)

Places one lightweight BusyBox Pod on every schedulable node. When a node joins the cluster, the DaemonSet controller adds a Pod to it; when a node leaves, its Pod is garbage-collected.

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: ahnaf-tripplanner-daemonset
  namespace: travel-dev
  labels:
    app: tripplanner-logs
    environment: development
spec:
  selector:
    matchLabels:
      app: tripplanner-logs
  template:
    metadata:
      labels:
        app: tripplanner-logs
        environment: development
    spec:
      containers:
      - name: log-collector
        image:  busybox:1.36
        command: ["/bin/sh","-c"]
        args:
          - "while true; do echo '[LOG] tripplanner log collector running on node'; sleep 60;
done"
        resources:
          requests:
            memory: "32Mi"
            cpu: "50m"
          limits:
            memory: "64Mi"
            cpu: "100m"
```

## Field-by-field explanation

| FIELD | VALUE | MEANING |
|---|---|---|
| `kind` | `DaemonSet` | There is no `replicas` field; the number of Pods equals the number of eligible nodes. |
| `selector` / labels | `app: tripplanner-logs` | Its own label, kept separate from the web and DB tiers. |
| `image` | `busybox:1.36` | Small image with a shell, suggested by the assignment. |
| `args` | `while true; do echo …; sleep 60; done` | A continuous process that prints a heartbeat every 60 seconds, so the container never exits. |
| `resources` | req 32Mi/50m · lim 64Mi/100m | Kept small because it runs on every node. |

| | | |
|---|---|---|
| tolerations | (none) | Nodes with a `NoSchedule` taint, typically the kubeadm control-plane node, are skipped. Those are not “eligible” nodes. |

**Partial: hostname not printed**

Requirement 3 asks the loop to print the **node hostname** and a heartbeat. The current message is a fixed string (“…running on node”) with no hostname or timestamp. A suggested Downward-API fix is in §12.

### Verify

```
kubectl apply -f 05-daemonset.yaml
kubectl get ds ahnaf-tripplanner-daemonset -n travel-dev      # DESIRED = CURRENT = node count
kubectl get pods -n travel-dev -l app=tripplanner-logs -o wide  # NODE column: one per node
kubectl logs -n travel-dev -l app=tripplanner-logs --tail=3
# requirement 6: add a node (e.g. minikube node add) and watch a new Pod appear
minikube node add && kubectl get pods -n travel-dev -l app=tripplanner-logs -o wide -w
```

06-job.yaml 07-cronjob.yaml QUESTIONS 6 & 7

# 10 Job & CronJob

Both files exist in the repository but are **empty (0 bytes)**, so Questions 6 and 7 are not yet implemented. Applying an empty file creates nothing. This section explains what each resource is responsible for and gives a suggested manifest that meets every listed requirement. **The suggested code is not in the repository.**

## 06-job.yaml as committed

This file is empty (0 bytes) in the repository.

## Suggested 06-job.yaml

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: ahnaf-tripplanner-cleanup
  namespace: travel-dev
  labels:
    app: tripplanner-cleanup
    environment: development
spec:
  backoffLimit: 3
  template:
    metadata:
      labels:
        app: tripplanner-cleanup
        environment: development
    spec:
      restartPolicy: Never
      containers:
        - name: cleanup
          image: busybox:1.36
          command: ["/bin/sh", "-c"]
          args:
            - |
              echo "[JOB] tripplanner DB cleanup started at $(date)"
              echo "Removing expired booking sessions..."; sleep 5
              echo "Purging stale itinerary drafts..."; sleep 5
              echo "[JOB] Cleanup completed successfully at $(date)"
          resources:
            requests:
              memory: "32Mi"
              cpu: "50m"
            limits:
              memory: "64Mi"
              cpu: "100m"
```

| REQUIREMENT | HOW THE SUGGESTION MEETS IT |
|---|---|
| busybox:1.36, shell script | `/bin/sh -c` with a multi-line script ( `\|` block scalar). |
| Start time + completion message | `echo … $(date)` at the start and end. |

| | |
|---|---|
| `backoffLimit` | `3`: retries up to 3 times before the Job is marked Failed. |
| `restartPolicy: Never` | A failed container is not restarted in place; the Job creates a new Pod instead, so failed Pods stay around for inspecting their logs. |

## 07-cronjob.yaml as committed

**This file is empty (0 bytes) in the repository.**

## Suggested 07-cronjob.yaml

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: ahnaf-tripplanner-scheduled-cleanup
  namespace: travel-dev
  labels:
    app: tripplanner-maintenance
    environment: development
spec:
  schedule: "*/5 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        metadata:
          labels:
            app: tripplanner-maintenance
            environment: development
        spec:
          restartPolicy: Never
          containers:
            - name: maintenance
              image: busybox:1.36
              command: ["/bin/sh", "-c"]
              args:
                - |
                  echo "[CRON] Maintenance run at $(date -u +%Y-%m-%dT%H:%M:%SZ)"
                  echo "VACUUM / ANALYZE simulated on tripplanner DB"; sleep 3
                  echo "[CRON] Maintenance finished"
```

| REQUIREMENT | HOW THE SUGGESTION MEETS IT |
|---|---|
| Every five minutes | `schedule: "*/5 * * * *"` (minute, hour, day, month, weekday). |
| Execution timestamp | `date -u +%Y-%m-%dT%H:%M:%SZ` prints an ISO-8601 UTC time. |
| History limits | Keeps the last 3 successful and 1 failed Job; older ones are deleted automatically. |
| `concurrencyPolicy` | `Forbid`: if the previous run is still going, the next one is skipped, so two maintenance runs never overlap. |

**Job: responsibility**

Run Pods until a set number finish successfully, then stop. Use it for one-off work such as migrations, cleanup or batch imports.

**CronJob: responsibility**

A Job factory on a clock. At each scheduled time it creates a new Job from `jobTemplate`. CronJob → Job → Pod.

### Verify

```
kubectl get jobs -n travel-dev                                  # COMPLETIONS 1/1
kubectl logs -n travel-dev job/ahnaf-tripplanner-cleanup
kubectl get cronjobs -n travel-dev                              # SCHEDULE */5 * * * *
kubectl get jobs -n travel-dev -w                               # new ahnaf-tripplanner-scheduled-
cleanup-<id> every 5 min
kubectl create job manual-run --from=cronjob/ahnaf-tripplanner-scheduled-cleanup -n travel-dev   #
test without waiting
```

# 11 Deployment order & verification

Apply in dependency order. Services can be created before their Pods; endpoints fill in once matching Pods become Ready. Creating the headless Service before the StatefulSet means Pod DNS works from the very first Pod.

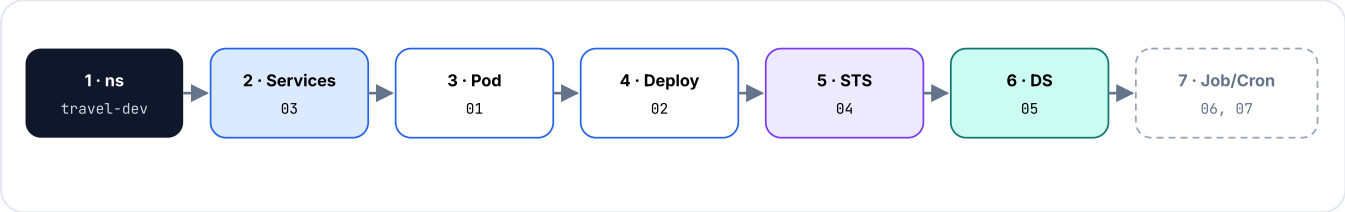



Figure 5. Recommended apply order.

```
kubectl create namespace travel-dev
kubectl apply -n travel-dev -f 03-service.yaml      # -n also catches the headless Service
kubectl apply -f 01-pod.yaml
kubectl apply -f 02-deployment.yaml
kubectl apply -f 04-statefulset.yaml
kubectl apply -f 05-daemonset.yaml
kubectl apply -f 06-job.yaml -f 07-cronjob.yaml   # once the files have content
# (or everything at once:  kubectl apply -n travel-dev -f .)
```

## Required screenshot commands (assignment §5)

| COMMAND | WHAT IT SHOULD SHOW FOR THIS PROJECT |
|---|---|
| `kubectl get all -n travel-dev` | The Pod, 3 Deployment Pods, the ReplicaSet, 2 DB Pods, DaemonSet Pods, both Services, the Deployment, StatefulSet and DaemonSet. |
| `kubectl get pods -n travel-dev -o wide` | All Pods Running, with Pod IPs and the node each one runs on. |
| `kubectl get deployments -n travel-dev` | `ahnaf-tripplanner-deployment 3/3` |
| `kubectl get services -n travel-dev` | NodePort `80:30097/TCP`; headless `None 5432/TCP` |
| `kubectl get statefulsets -n travel-dev` | `ahnaf-tripplanner-db 2/2` |
| `kubectl get daemonsets -n travel-dev` | DESIRED = CURRENT = READY = number of eligible nodes |
| `kubectl get jobs -n travel-dev` | Cleanup Job `1/1`, plus CronJob-spawned Jobs (needs 06/07) |
| `kubectl get cronjobs -n travel-dev` | `*/5 * * * *`, LAST SCHEDULE updating (needs 07) |
| `kubectl get pvc -n travel-dev` | `data-ahnaf-tripplanner-db-0` and `-1`, both `Bound`, 1Gi, RWO |

## Resource budget

| WORKLOAD | PODS | CPU REQUEST | CPU LIMIT | MEM REQUEST | MEM LIMIT |
|---|---|---|---|---|---|
| Pod (01) | 1 | 250m | 500m | 64Mi | 128Mi |

| | | | | | |
|---|---|---|---|---|---|
| Deployment (02) | 3 | 3 × 100m = 300m | 1500m | 192Mi | 384Mi |
| StatefulSet (04) | 2 | 200m | 500m | 256Mi | 512Mi |
| DaemonSet (05) | N nodes | N × 50m | N × 100m | N × 32Mi | N × 64Mi |
| **Total (1 node)** | **7** | **800m** | **2600m** | **544Mi** | **1088Mi** |

Plus 2 × 1Gi of persistent storage. This fits comfortably on a default minikube or kind node (2 CPU / 2 GiB or more).

## 12 Compliance review & recommended fixes

Each requirement is checked against the manifests as committed. MET means it is satisfied by the YAML. DEMO means it must be shown on a live cluster with a screenshot. PARTIAL and MISSING need a code change.

| Q | REQUIREMENT | STATUS |
|---|---|---|
| 1 | nginx:1.27 · unique name · port 8017 · app/environment labels · requests & limits | MET |
| 1 | Pod reaches Running | DEMO |
| 2 | nginx:1.27 · 3 replicas · labels & selector · port · resources | MET |
| 2 | Scaling operation · rolling update | DEMO |
| 3 | NodePort 30097 · port 80 / targetPort 8017 · selector · headless Service in same file | MET |
| 3 | Headless Service namespace set to `travel-dev` | PARTIAL |
| 3 | Accessibility through the Service | DEMO |
| 4 | postgres:16 · 2 replicas · unique name · serviceName · volumeClaimTemplates · PVC per replica | MET |
| 4 | Predictable names · delete & recreate · persistence | DEMO |
| 4 | Credentials from a Kubernetes Secret | MISSING |
| 5 | DaemonSet · busybox:1.36 · continuous heartbeat · labels | MET |
| 5 | Prints the node hostname | PARTIAL |
| 5 | One Pod per node · add-node behaviour | DEMO |
| 6 | Job: busybox · cleanup script · timestamps · backoffLimit · restartPolicy Never | MISSING |
| 7 | CronJob: */5 schedule · timestamp · history limits · concurrencyPolicy | MISSING |

### Suggested fix A: Secret for PostgreSQL credentials (04-statefulset.yaml)

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: ahnaf-tripplanner-db-secret
  namespace: travel-dev
type: Opaque
stringData:
  POSTGRES_DB: tripplanner
  POSTGRES_USER: tripplanner
  POSTGRES_PASSWORD: change-me
---
# then, inside the StatefulSet container, replace the 3 plain env vars with:
          envFrom:
            - secretRef:
                name: ahnaf-tripplanner-db-secret
```

### Suggested fix B: add the namespace to the headless Service (03-service.yaml, document 2)

```yaml
metadata:
  name: ahnaf-tripplanner-db-service
  namespace: travel-dev
  labels:
    app: tripplanner-db
```

### Suggested fix C: print the node hostname in the DaemonSet (05-daemonset.yaml)

The Downward API injects the name of the node the Pod was scheduled on. `$(hostname)` inside a container returns the *Pod* name, not the node name, so both are printed.

```yaml
        env:
          - name: NODE_NAME
            valueFrom:
              fieldRef:
                fieldPath: spec.nodeName
        args:
          - while true; do echo "[HEARTBEAT] $(date) node=${NODE_NAME} pod=$(hostname)"; sleep
60; done
```

### Minor observations (no action required)

- **Standalone Pod is a Service backend.** Because `01-pod.yaml` carries `app=tripplanner`, the NodePort Service balances across 4 Pods, not 3. That is harmless, but when you show the endpoints, explain why there are 4 IPs.
- **Deployment Pods lack the `environment` label.** It is only on the Deployment object (see §6).
- **Database name "postgress".** The doubled *s* is valid but looks like a typo. If you change it, use the same value in every `psql -U/-d` command.
- **Old PVCs persist.** Deleting the StatefulSet keeps `data-ahnaf-tripplanner-db-*`. That is intentional, so the data survives. Remove them explicitly with `kubectl delete pvc data-ahnaf-tripplanner-db-0 data-ahnaf-tripplanner-db-1 -n travel-dev`.
- **Submission packaging.** The assignment expects a folder named like `student-17-tripplanner/`, zipped as `student-17-tripplanner.zip`, with execution screenshots included.